MOBILE APPLICATION DVELOPMENT

Shoaib Farooq

Department of Computer Science

Instructor Information

Instructor	Shoaib Farooq	E-mail	m.shoaib1050@gmail.com
 GitHub	https://github.com/SHOAIB1050		
 YouTube	https://www.youtube.com/c/pakithub		
 LinkedIn	https://www.linkedin.com/in/shoaib-farooq-b5b190105/		

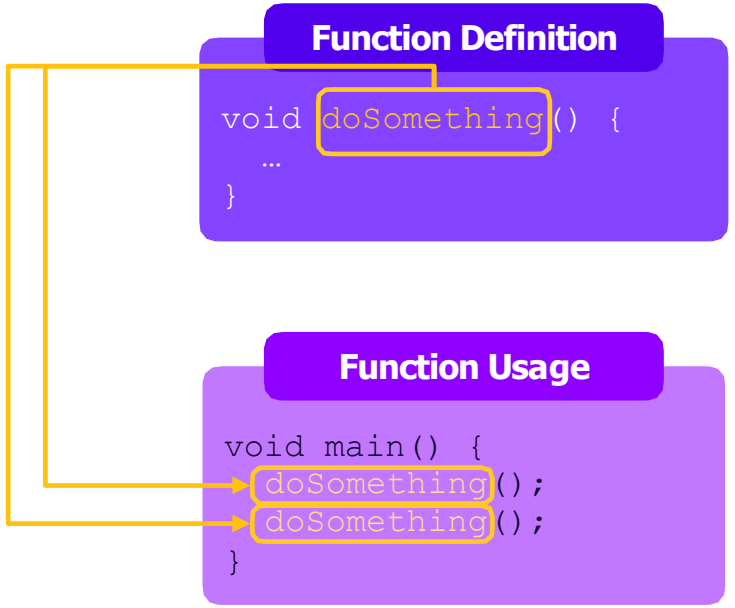
Let's Start .....

Lecture # 4,5

OUTLINE

- Functions
- Functions & Parameters
- Named Vs Positional Argument
- “Final” Vs “Const” Vs “Var”
- Understanding Classes
- Widgets Are Complex Objects
- Column & Row
- Functions As Values
- Customized Based Widgets

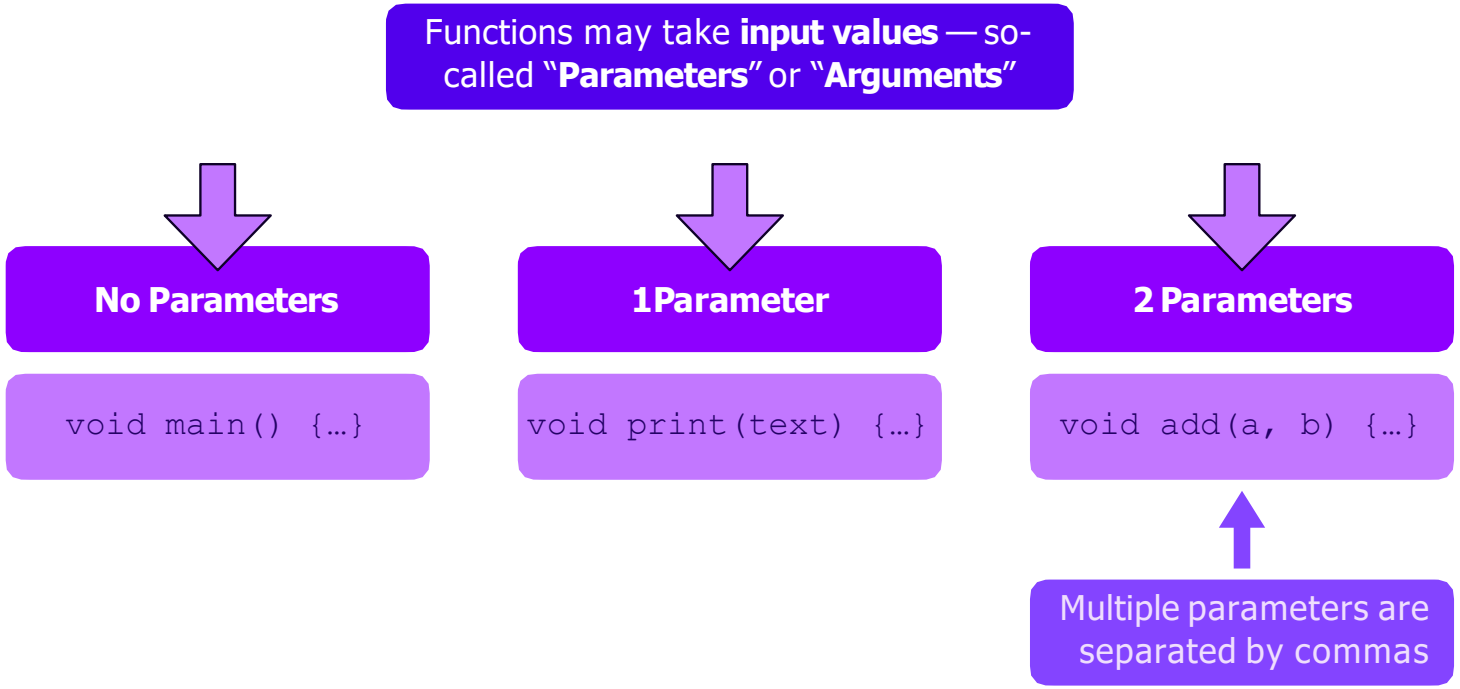
FUNCTIONS



doSomething is the **function name** (i.e., it's **identifier**)

Functions can be used ("called") in your code **as often as needed**

FUNCTIONS & PARAMETERS



these are just examples — functions can accept as many arguments / parameters as needed)

NAMED VS POSITIONAL ARGUMENTS

Functions may receive input values as "**named**" or "**positional**" **arguments**

Named Arguments

add(**num1**: 1, **num2**: 5)

Names of arguments must be specified to pass values to those arguments

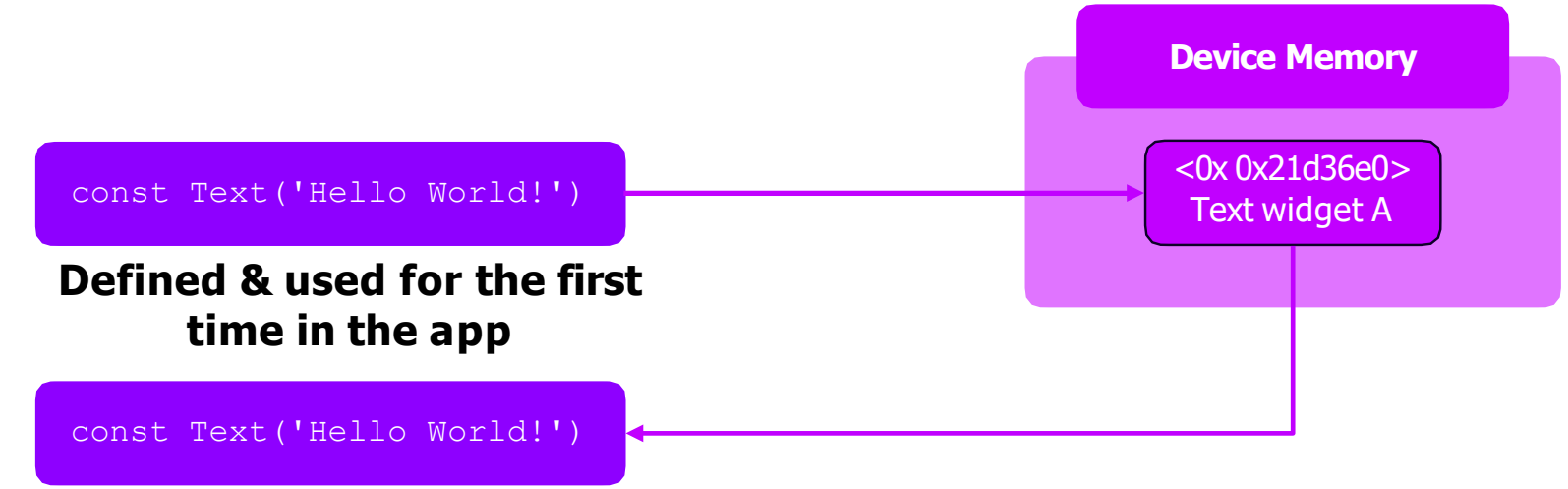
Positional Arguments

add(1, 5)

Argument values are mapped by position (first expected parameter receives first argument etc.)

UNDERSTANDING “CONST”

const helps Dart optimize runtime performance



“FINAL” VS “CONST” VS “VAR”

var:

- var is a keyword used for variable declaration without specifying the type explicitly. The type is inferred at compile-time based on the assigned value.
- The variable's type can change based on the assigned value.
- var myVar = "Hello, Flutter!"; // Inferred as String

final:

- final is a keyword used to declare a variable that can be assigned a value only once. Once assigned, the value cannot be changed.
- It is finalized at run time.
- The type of the variable is determined either explicitly or clear from the assigned value.
- final int myNumber = 42;

const:

- const is a keyword used to declare a compile-time constant. The value must be known at compile-time, and it cannot change during runtime.
- const variables are implicitly final, but they are also required to be assigned a value that is a constant expression.

const double piValue = ?;

“FINAL” VS “CONST” VS “VAR”

`var`

Creates a new variable that will be re-assigned at some point

Use the type (e.g., `String`) instead of var if the variable has no initial value

Otherwise, the type can be inferred by Dart

`final`

Create a new run- time constant

Will (and can) never be re-assigned

`const`

Create a new compile-time constant

Will (and can) never be re-assigned & value is “hardcoded” (fixed) at compile-time

Can’t be used if some code must be executed in order to derive the value

UNDERSTANDING CLASSES

Dart is an object-oriented language

Every value is an **object**

Primitive Values

Text
`'Hello World'`

Numbers
`30, 12.31`

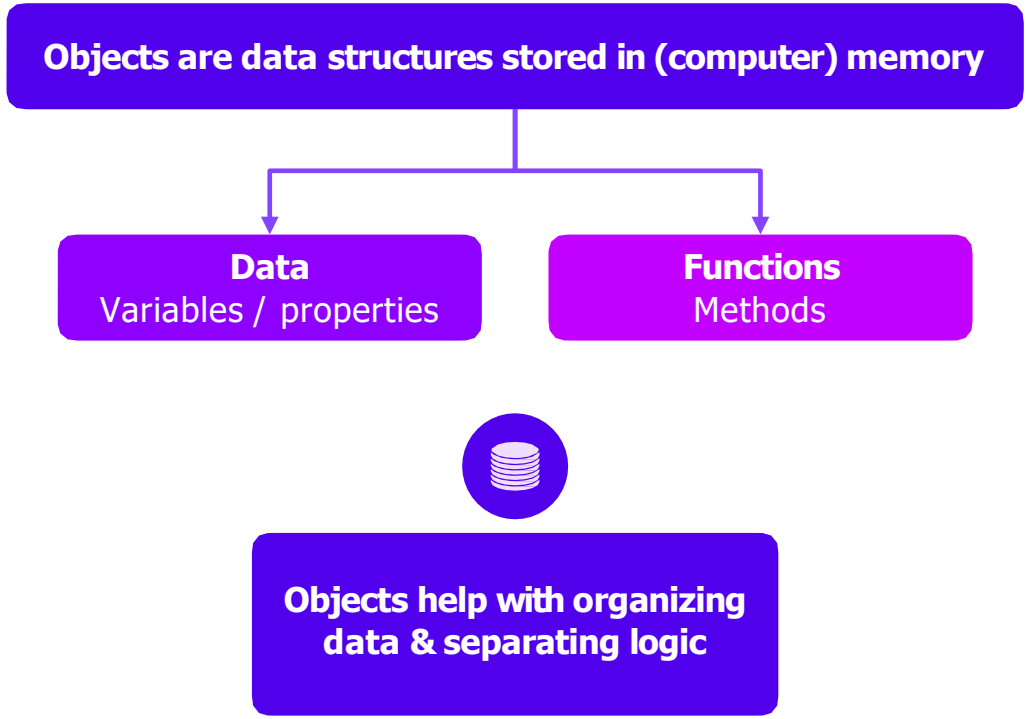
...

More Complex Values

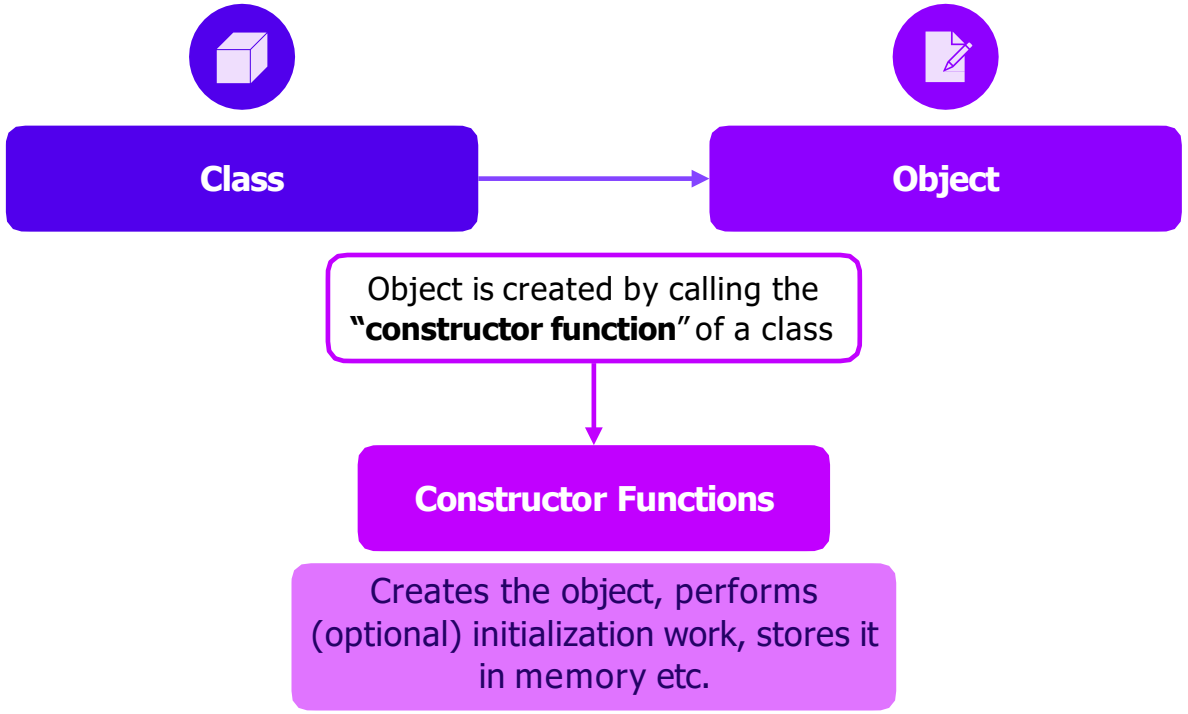
e.g., Widgets, Gradient Config Object

Created based on blueprints: **Classes**

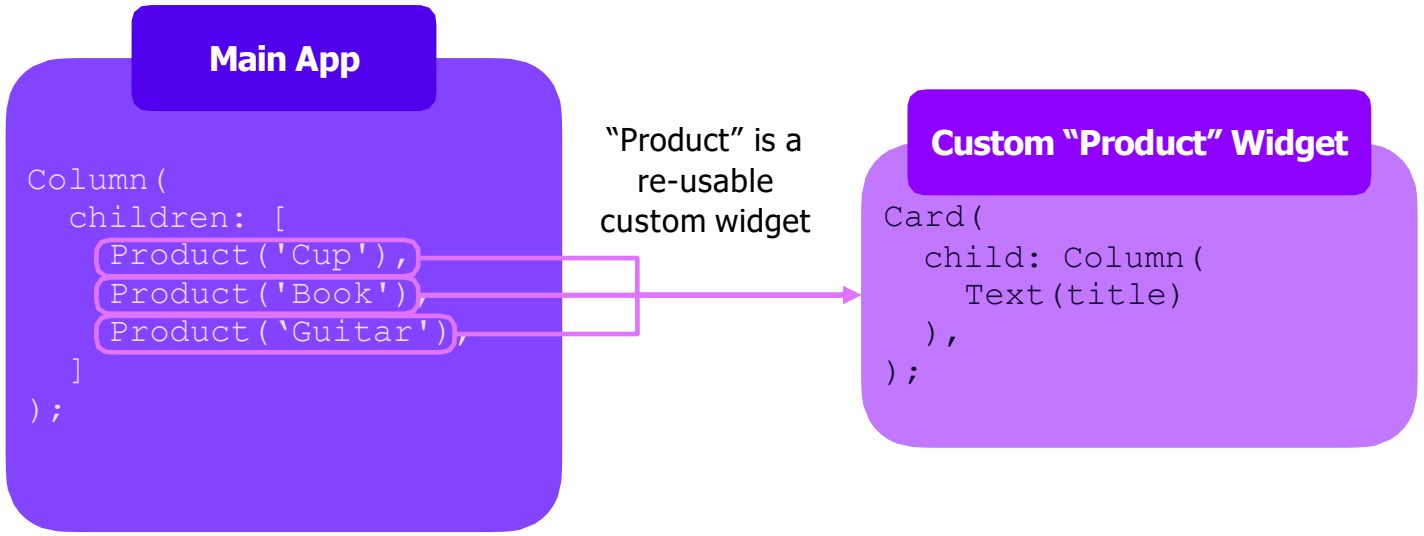
OBJECTS = DATA STRUCTURES



OBJECTS ARE CONSTRUCTED FROM CLASSES

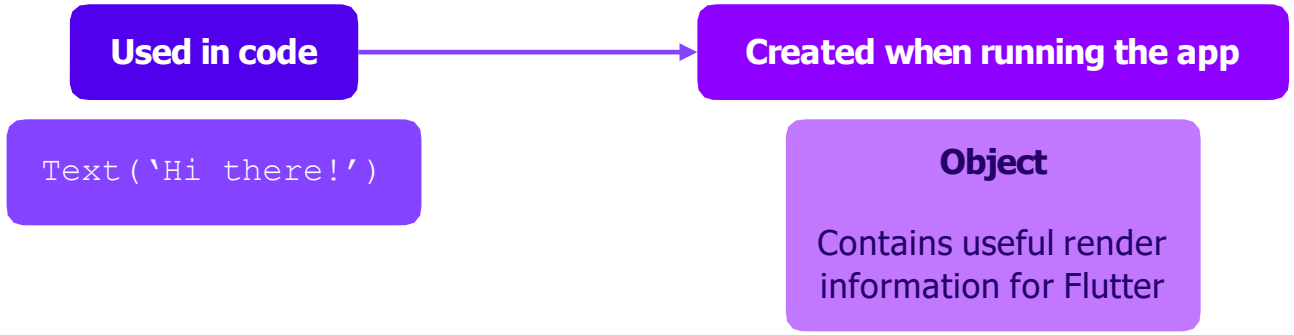


BUILDING CUSTOM WIDGETS



(of course, Column, Card & many other built-in widgets are explained throughout the course)

WIDGETS ARE COMPLEX OBJECTS



COLUMN & ROW

`Column()` & `Row()` can be used to place **multiple child widgets next to each other**

`Column()`

Main Axis: **Vertical** Axis

Cross Axis: Horizontal Axis

By default, occupies the **entire available height** but **only the width required** by its content (children)

`Row()`

Main Axis: **Horizontal** Axis

Cross Axis: Vertical Axis

By default, occupies the **entire available width** but **only the height required** by its content (children)

FUNCTIONS AS VALUES

In Dart, functions are also just values / objects!

Defining a Function

```
void start() { … }
```

Code inside the function **does not yet execute** — instead it's defined for **execution in the future**

Calling a Function

```
start();
```

Code defined inside the function **is executed**

Using a Function as a Value

```
TextButton(onPressed: start)
```

Code inside the function is **not yet executed** — instead the function **may be called from inside the receiving object** (e.g., widget)

CUSTOMIZED BASED WIDGETS

STATELESS VS STATEFUL WIDGETS

Stateful Widget:

- A stateful widget in Flutter is a widget that can change its internal state during its lifetime. It can rebuild itself in response to user interactions or other events.
- Stateful widgets are useful when you have parts of your user interface that can change dynamically and need to be updated.

Stateless Widget:

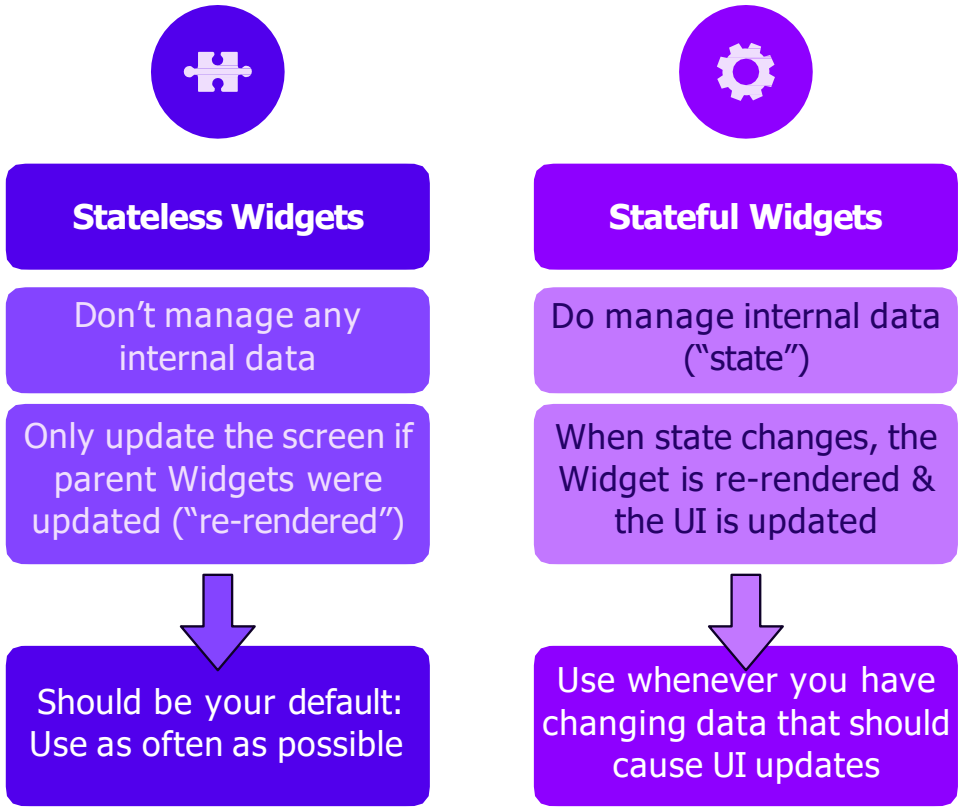
- A stateless widget is a widget that does not change its internal state during its lifetime. Once a stateless widget is built, it cannot be rebuilt.
- Stateless widgets are useful when the part of your user interface does not change based on user interactions or other events.

INHERITED WIDGET

- InheritedWidget is a special type of widget in Flutter that allows data to be passed down the widget tree to its descendants. It is particularly useful when you have data that needs to be accessible by multiple widgets without explicitly passing it to each one.

- It helps in managing and propagating state efficiently through the widget tree.

STATELESS VS STATEFUL WIDGETS



Function & Method

Difference

A **function** is a piece of code that is called by name

It is not associated with an any object.

Data passed to a function is explicitly.

A **method** is a piece of code that is called by a name

It is associated with an object.

A method is implicitly passed the object on which it was called

A method is able to operate on data that is contained within the class

- **F**unction → **F**ree (Free means not belong to an object or class)
- **M**ethod → **M**ember (A member of an object or class)

Function & Method

Difference

```cpp
void myFunction() {
  cout << "I just got executed!";
}

int main() {
  myFunction(); // call the function
  return 0;
}
```

```java
public class MyClass {
  // Static method
  static void myStaticMethod() {
    System.out.println("Static methods can be called without creating objects");
  }

  // Public method
  public void myPublicMethod() {
    System.out.println("Public methods must be called by creating objects");
  }

  // Main method
  public static void main(String[] args) {
    myStaticMethod(); // Call the static method
    // myPublicMethod(); This would compile an error

    MyClass myObj = new MyClass(); // Create an object of MyClass
    myObj.myPublicMethod(); // Call the public method on the object
  }
}
```

MODULE SUMMARY

Starting Flutter Apps

main.dart → main() → runApp()

Widgets, Widgets, Widgets

Flutter UIs are built by combining widgets

Widgets are nested into each other ("widget tree")

Core Dart Features

Types, functions, variables, classes, objects, …

Let the code editor (e.g., VS Code) help you!

Custom Widgets

StatelessWidget doesn't change internally

StatefulWidget updates the UI upon state changes

Configuring Widgets

Many (built-in) widgets offer (named) configuration arguments

Typically, configuration objects are used

Thank You ☺